

RedrobRank Agentic Recruiter OS

Problem

Keyword filters miss real fit. Traditional ATS systems rely on exact match parsing and fail to ground extracted signals in real production or evaluation contexts.

Role understanding

Senior AI Engineer founding team. The JD Agent parses this into structured needs: must-haves (Core AI, Production, Evaluation, Python), nice-to-haves (MLOps, Leadership), and explicit constraints around seniority and location.

Agent Architecture

1. JD Agent: Parses job description.
2. Evidence Agent: Extracts grounded evidence from profiles.
3. Ranking Agent: Deterministically scores candidates.
4. Risk Agent: Identifies honeypots and bad fit.
5. HITL Agent: Enforces recruiter approval.

Feature Scoring

Weighted deterministically based on: Core AI (35%), Production shipping (18%), Evaluation metrics (14%), Python architectures (10%), Career shape (8%), Behavioral signals (7%), and Location (4%).

Trap Defense

We penalize candidates and flag them for recruiters if they trigger honeypot rules, keyword stuffing rules, impossible claims (e.g. expert proficiency in 3 months), or YOE mismatches.

Human-in-the-loop

Recruiter approval is strictly required before outreach. The UI pauses the automated agent workflow at the shortlist phase to provide a trust barrier.

Explainability

Every rank is paired with grounded evidence and explicit audit reports detailing how and where keywords and evaluation signals were matched in the profile.

Reproducibility

Designed for CPU-only environments. Zero network dependencies (LLMs). Real-time runtime execution guaranteed under resource constraints and validated strictly by the required pipeline.

Final Output

The pipeline exports the top 100 CSV alongside a recruiter packet flow that locks further automation until human sign-off.